

Mesh partitioning and node ordering for FESOM2/Kokkos: what a decomposition is worth on CPU and GPU

Campaign report

12 August 2026 — Levante (128-core CPU nodes, 4×A100 GPU nodes; GH200 testbed)

1. Scope

FESOM2 reads its domain decomposition from files (`dist_N`), so a better partition is a drop-in change: no model code, no recompilation, no effect on any other configuration. This campaign asked what that degree of freedom is worth for the C++/Kokkos port, on both backends, and whether the node *numbering* of the mesh (which fixes memory-access locality) is worth changing as well.

The short answer: on GPU the shipped partitions leave 8–20 % of the model step on the table, and the single most valuable knob (MINCONN) has never been active in any FESOM partition ever generated. On CPU the gains are smaller (4–8 %) and come from a different place — external partitioning engines and load-balance slack, not MINCONN. The two backends want opposite things from a partition, and one recipe does not serve both.

The campaign also produced a result we did not go looking for: **repartitioning these meshes occasionally produces partitions on which the model diverges**, with any recipe — including the stock one — and no offline metric predicts which. Eight such partitions were caught. This turns a stability screen from a formality into the load-bearing part of any adoption procedure, and it is why every number below is labelled with its evidence.

Meshes: CORE2 (127k surface nodes), fArc (638k), DARS (3.16M), NG5 (7.40M). All runs are cold starts at each mesh’s stability-ladder time step (CORE2 1800 s, fArc 900 s, DARS 120 s, NG5 180 s), JRA55 1958 forcing, PHC initial state, frozen certified binaries.

2. What was tried

METIS options. FESOM calls `METIS_PartGraphRecursive`, which silently accepts and ignores three options: the communication-volume objective (`OBJTYPE=VOL`), contiguity (`CONTIG`) and MINCONN (minimise the maximum number of neighbouring subdomains). We switched the call to `PartGraphKway` and swept: the objective, MINCONN, `CONTIG`, load-balance slack (`UFACTOR 10/30/100`), scalar vertical weights $w = a + nlev$ for $a \in \{0, 15, 40, 100\}$, hierarchical two-level partitions, and METIS seeds. To be fair to the original authors: the Recursive choice is documented in `fort_part.c` with a measured justification on one mesh; what has changed since is the hardware.

External engines. KaHIP, KaMinPar and Mt-KaHyPar, each fed the same node-weighted dual graph, under the same scalar weight sweep.

Node renumbering. Hilbert space-filling-curve reordering of the mesh files themselves (a deeper change than a `dist_N` swap: every reference run pinned to the old numbering must be re-baselined), plus reverse Cuthill–McKee.

Cross-architecture check. The winning GPU settings were re-raced on DKRZ’s GH200 partition (dolpung), whose fabric has no GPUDirect and stages every halo through pinned host memory — a deliberately different per-message cost structure.

3. How things were measured

Speed numbers come from *rac*es: all arms in one SLURM allocation, interleaved repetitions, 300 steps, min-of- N against the shipped partition of the same mesh, at the ladder time step. The five mesh-definition files are md5-verified identical across arms before any leg runs, so arms differ only in the partition. Offline partition metrics (edge cut, halo volume, partner counts, imbalance) were computed for every candidate — and used only for design, never as a gate (Section 5).

A raced gain is not a result until it passes two gates:

- **Stability screen: 3,000 steps at the target rank count**, return code and trajectory checked. This bar exists because a 150-step winner on DARS (MINCONN, -4.5%) diverged between step 150 and step 3,000 while its baseline ran clean.
- **Accuracy gate: the control envelope**. Twenty steps with per-step output; the arm’s temperature, salinity and SSH rms differences against base must sit inside the spread of at least three *seed controls* — re-rolls of the shipped recipe differing only in the METIS seed, each verified to be a genuinely different partition. Any partition change perturbs the state at round-off and the differences grow chaotically at fronts, so “zero difference” is not available; “indistinguishable from an ordinary repartitioning” is the operative standard. The rms-inside-the-envelope yardstick is fixed campaign policy; quantiles are reported as context but never decide.

Certified partitions leave the campaign only through a promotion tool that refuses any `dist_N` lacking a recorded 3,000-step clean run at exactly that rank count.

4. Results

4.1. The board

mesh	backend, ranks	gain	setting	stability	accuracy
DARS	GPU, 64	-19.7%	MINCONN+CONTIG+u30	✓ 3,000 steps	pending ^a
NG5	GPU, 64	-9.8%	MINCONN	✓	pending ^a
CORE2	GPU, 4	-8.1%	MINCONN	✓	✓
fArc	GPU, 16	-3.6%	MINCONN+CONTIG	✓	failed: do not ship
fArc	CPU, 2048	-7.5%	Mt-KaHyPar $w=100+nlev$	✓	✓ (4 controls)
CORE2	CPU, 512	-5.8%	Hilbert + engine	✓	✓
DARS	CPU, 2048	-4.2%	KaMinPar $w=100+nlev$	✓	✓
CORE2	CPU, 864	-4.9%	KaMinPar	✓	failed (salt): do not ship
CORE2	CPU, 512	-3.8%	UFACTOR=30 alone	✓	✓
NG5	CPU, 2048	null	—	—	—

Table 1: Best raced arm per point, measured against the shipped partition at the ladder time step, after protocol-length re-measurement. ^aAt the time of writing the DARS and NG5 GPU accuracy gates are in the queue; the DARS first pass put all arms just outside a 3-control envelope and the verdict awaits five controls (Section 4.4).

Two regularities. First, **every gain re-measured at 3,000 steps came back equal to or larger than its 300-step race value** (eight matched pairs; e.g. DARS GPU $-18.6 \rightarrow -19.7\%$, CORE2 GPU $-7.5 \rightarrow -8.1\%$): the short race systematically under-reports, so these numbers are conservative. Second, the backend split is consistent everywhere: the two knobs (CONTIG+slack) that gain five points on DARS GPU *cost* a point on DARS CPU.

4.2. Why: the GPU pays per message, the CPU pays for owned work

Across all nine raced groups, the number of communication partners is the only offline metric family that correlates positively with GPU step time — every volume-like metric (edge cut, halo bytes, replicated elements) correlates *negatively*: arms that ship more data are faster when they talk to fewer neighbours. MINCONN is the only METIS objective that targets partner count, which is why it wins on GPU and why the win was invisible for as long as the Recursive call ignored it.

Across meshes, the size of the GPU gain tracks the *fractional reduction in the maximum partner count* (Table 2) — the max, not the mean, because the step waits for the slowest rank. By mean partner count the three points are indistinguishable.

On CPU no offline metric orders the arms — not imbalance, not edge cut, not partner count (median within-group Spearman ρ straddles zero for every column). The often-quoted imbalance correlation ($r = 0.91$) is a statement about where time goes *within* one partition, not about which partition is

point	base $\text{nbr}_{\max}$	arm $\text{nbr}_{\max}$	reduction	GPU gain
fArc 16	7	6	−14 %	−3.6 %
CORE2 4	3	2	−33 %	−8.1 %
DARS 64	12	7	−42 %	−19.7 %

Table 2: The GPU gain is ordered, roughly proportionally, by the fractional reduction of the maximum per-rank partner count. Valid within one machine; the coefficient is fabric-dependent (Section 4.5).

faster. On CPU you have to race. What does generalise is that the external engines win at every large-CPU point, and are simultaneously the best-behaved arms in the accuracy gates.

4.3. Node renumbering: CORE2 only

Hilbert renumbering is worth −1.2 to −2.4 % (CPU) and −5.0 % (GPU, one node) on CORE2 and is $\sim 94\%$ additive with repartitioning (−5.8 % combined at 512 ranks). It is worth nothing on any other mesh: fArc, DARS and NG5 already ship at 88 % element-gather locality against CORE2’s 27.6 %. Because renumbering rewrites the mesh files — forcing every pinned reference run to be re-baselined — we recommend it only where that disruption is priced in; the repartitioning gain is a pure `dist_N` swap and carries no such cost.

4.4. Accuracy: the control envelope did real work

The gates are a comparison against ordinary repartitionings, and they discriminated in both directions. At CORE2 4 GPU the winners sit *below* every control; at DARS 2048 CPU KaMinPar sits below every control on temperature and salinity. Conversely fArc 16 GPU failed (both arms above a four-control temperature envelope) and CORE2 864 failed on salt against five controls — both points are speed wins that we do not recommend. Control spreads vary by an order of magnitude between meshes ($\times 1.05$ on DARS, $\times 3.9$ on CORE2 at 864), so a fixed rms threshold would be meaningless; the per-mesh envelope is what makes the verdicts comparable.

One mechanism deserves its own sentence: partitions that make the SSH solver converge *faster* systematically show larger SSH differences, because the solver stops on a relative residual — stopping sooner lands further from the reference. This inflates the SSH statistic for exactly the partitions one wants; it is visible at CORE2 and DARS and is part of why the DARS GPU gate is being decided on a widened envelope rather than a first pass.

4.5. Cross-architecture: the lever is real off-A100, but only proven at one node

On the GH200 testbed the CORE2 single-node result reproduces and slightly exceeds A100 (MINCONN −8.9 vs −8.1 %; two independent jobs within 0.6 pp). A pre-registered prediction that DARS 64 (16 GH200 nodes) would beat its A100 −18.6 % **failed**: the measured effect is −1 to −3 %, and fArc (4 nodes) is a clean null at min-of-5. Once the halo leaves pinned host on the inter-node fabric, the per-message premium largely vanishes on that machine. Conclusions: the lever is not an A100 artefact (different silicon, different transport, same win at one node), but multi-node gains must be measured per fabric, never extrapolated — and min-of- N is a biased estimator whenever the baseline is noisier than the arms (on dolpung it always was; medians must be reported alongside).

5. What we learned beyond the numbers

1. Repartitioning occasionally produces fatal partitions, with any recipe. Eight partitions caught so far: MINCONN-family and slack-only arms on NG5 and DARS at both 64 and 2048 ranks — and one *seed-only re-roll of the stock shipped recipe*, which blew up at step 14. The failure is always local (one spot runs away while global diagnostics stay sane for tens of steps) and has no offline signature: the dying partitions score *better* than survivors on every scorecard column we computed. The shipped partitions in production are, in this precise sense, lucky seeds.

2. The remedy is cheap and verified: re-roll the seed. On both meshes where a winner died, regenerating the identical arm with a different METIS seed produced a partition that passed the full screen (NG5, DARS). A failure is a lottery ticket, not a verdict on the knobs.

3. Short tests prove nothing about stability. A 5-step smoke passes on a partition that dies at step 71; a 150-step race passes on one that dies before step 3,000. Screen length is part of any claim’s identity, and the 3,000-step floor caught what would otherwise have been shipped twice.

4. Accuracy needs an envelope, not a threshold or a single control. One control condemned an innocent arm (its class spans $\times 2.7$ – 3.9 between seeds); a control that coincides with the base partition measures round-off and condemns everything. At least three controls, each demonstrated distinct, before any verdict.

5. Protocol discipline is what the numbers rest on. Cold starts must run at the ladder time step (four separate incidents of racing at production Δt cost three findings and, once, the baseline itself); `srun: task Killed` is teardown noise — the real error is hundreds of lines earlier; and a pre-registered prediction that fails (Section 4.5) is worth more than a post-hoc explanation that fits.

6. Recommendations

Upstream (FESOM2), in order of value:

1. Switch `fort_part.c` to `METIS_PartGraphKway` so `MINCONN`, `CONTIG` and `OBJTYPE=VOL` actually reach METIS, and expose the partitioner options as runtime switches rather than recompilation.
2. Ship the adoption procedure with it: **every newly generated partition — any engine, any options, any seed — must run $\geq 3,000$ steps at the target rank count at the mesh’s cold-start time step before use; on failure, re-roll the seed and screen again.** This is not a tax on the new options: the stock recipe fails it too.
3. The isolated-node repair in `check_partitioning` currently seeds its candidate list with a single neighbour; build the full list. Update to METIS 5.2.1.

Per-machine settings measured here (Levante): GPU runs — `MINCONN` on, the further knobs (`CONTIG`, slack) decided per point by a race, since they gain five points on DARS and cost elsewhere; CPU runs — an external engine (KaMinPar/Mt-KaHyPar) with $w = 100 + \text{nlev}$ where available, else `UFACTOR=30` slack; Hilbert renumbering on CORE2-class meshes only, and only where re-baselining is acceptable.

What not to do: do not gate a partition on the offline scorecard (it ranks dying partitions above surviving ones); do not certify with a smoke test; do not transfer multi-node GPU gains across fabrics without measuring.

7. Status and open items

The DARS 64 and NG5 64 GPU accuracy gates are in the queue at the time of writing; DARS’s first pass sits just outside a three-control envelope on all fields and is being re-judged against five controls, with the SSH-stopping mechanism (Section 4.4) as recorded context. If the -19.7% arm remains out of class, the fallback positions are explicit: `MINCONN` alone (-14.3% , near-in-class on first pass), or adoption as a documented exception — the gate verdict stays as measured either way. GH200 multi-node behaviour, and the question of whether a fabric with GPUDirect (JUPITER) prices messages like the A100 or like `dolpung`, are open and should be answered by racing, not by analogy.

Evidence: `docs/PARTITIONING_M11.md` (Findings 1–45, job ids for every number); one-page operational summary: `docs/M11_RECOMMENDATION.md`; race table: `scripts/m11_harvest_races.py -best`.